


**Università degli Studi di Milano-Bicocca**

Manuel Roscioli

Matricola**:** 868801

# PROGETTO METODI INFORMATICI PER LA GESTIONE AZIENDALE:

# Recommendation System basato sui dati di vestibilità dell'abbigliamento: Studio sul dataset Marketing Bias (ModCloth)

## Indice


**1 Analisi esplorativa 3**

1.1 Analisi variabili quantitative 4
1.2 Creazione dataset ridotto 4
1.3 Analisi della tendenza centrale dei rating 5
1.4 Analisi della dispersione dei rating 6
1.5 Analisi dei rating nelle quattro categorie 7
1.6 Categorie con i rating più alti e più bassi 9
1.7 Analisi categoria con il valore medio dei rating minore (Bottoms) 9
1.8 Valutazione dei rating medi per ogni articolo 10

**2 Configurazione ottimale dell' algoritmo k-nn per la predizione dei rating 10**

2.1 Creazione dataset ridotto 11
2.2 Grid Search 12
2.3 Predizione algoritmo KNNBasic 14

**3 Filling della matrice di rating con la configurazione ottimale 16**

3.1 Matrice di rating con gli 0 17
3.2 Matrice di rating con valori noti e previsioni 18

**4 Segmentazione utenti in base alle preferenze: K-Means (Cosine Similarity) 18**

4.1 Determinazione del numero k di cluster con Metodo Elbow 19
4.2 Valori WCSS e primo grafico usando matrice_kmeans_mid 20
4.3 Valori WCSS e grafico a gomito usando rating_matrix 21
4.4 Eseguo il K-Means (k=2 cluster) 22

**5 Creazione per ogni utente della lista degli n items da consigliare 23**

**6 Filling della matrice di rating attraverso Matrix Factorization (MF) 24**

6.1 Grid Search 2.0 24
6.2 Trainset e addestramento del modello SVD 25
6.3 Riempimento della matrice di rating (MF) 25

**CONCLUSIONI** 26

## *INTRODUZIONE*

Nella vasta gamma di capi di abbigliamento, scegliere quelli perfetti può risultare difficile e proprio per questo, il nostro progetto è incentrato sulla creazione di un Recommendation System per gli utenti di ModCloth.

ModCloth è un noto venditore di abbigliamento online che è incentrato sulla fiducia e sulla soddisfazione dei clienti tramite la vendita di capi che si adattino alla perfezione alle esigenze del venditore.

Il dataset studiato è Marketing Bias e contiene una vasta gamma di misurazioni, come taglie, lunghezze, misure del busto, della vita e dei fianchi, oltre ai feedback dei clienti riguardanti le preferenze e le opinioni sulla vestibilità. Analizzare questi dati è di cruciale importanza per i designer, i rivenditori e gli esperti di moda per creare e offrire consigli personalizzati ai clienti e per soddisfare le loro esigenze.

## 1. *ANALISI ESPLORATIVA*

| | item_id | user_id | rating | timestamp | size | fit | user_attr | model_attr | category | brand | year | split |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 7443 | Alex | 4 | 2010-01-21 08:00:00+00:00 | NaN | NaN | Small | Small | Dresses | NaN | 2012 | 0 |
| 1 | 7443 | carolyn.agan | 3 | 2010-01-27 08:00:00+00:00 | NaN | NaN | NaN | Small | Dresses | NaN | 2012 | 0 |
| 2 | 7443 | Robyn | 4 | 2010-01-29 08:00:00+00:00 | NaN | NaN | Small | Small | Dresses | NaN | 2012 | 0 |
| 3 | 7443 | De | 4 | 2010-02-13 08:00:00+00:00 | NaN | NaN | NaN | Small | Dresses | NaN | 2012 | 0 |
| 4 | 7443 | tasha | 4 | 2010-02-18 08:00:00+00:00 | NaN | NaN | Small | Small | Dresses | NaN | 2012 | 0 |

Il dataset è composto da 99893 righe e 12 colonne. Queste di seguito sono le variabili:

- Item_id: ID dell'articolo
- User_id: ID dell'utente che ha lasciato il rating
- Rating: rating assegnato (valore da 1 a 5)
- Timestamp: data in cui è stato registrato il rating
- Size: taglia dell'articolo
- Fit: vestibilità dell'articolo
- User_attr: caratteristiche del cliente
- Model_attr: caratteristiche del capo di abbigliamento
- Category: categoria dell'articolo
- Brand: il marchio dell'articolo
- Year: anno del capo

Inizialmente il dataset presentava dei valori mancanti che sono stati rimossi:

```
dfc.dropna().head()
```

| | item_id | user_id | rating | timestamp | size | fit | user_attr | model_attr | category | brand | year | split |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 328 | 21296 | Nikki | 5 | 2011-02-16 08:00:00+00:00 | 3.0 | Just right | Small | Small | Bottoms | ModCloth | 2013 | 0 |
| 329 | 21296 | ellen | 5 | 2011-02-16 08:00:00+00:00 | 1.0 | Just right | Large | Small | Bottoms | ModCloth | 2013 | 0 |
| 340 | 21296 | Sarita | 5 | 2011-02-22 08:00:00+00:00 | 2.0 | Just right | Small | Small | Bottoms | ModCloth | 2013 | 0 |
| 351 | 21296 | kacikorn | 4 | 2011-03-01 08:00:00+00:00 | 1.0 | Just right | Small | Small | Bottoms | ModCloth | 2013 | 0 |
| 361 | 21296 | lesleydawn88 | 4 | 2011-03-06 08:00:00+00:00 | 1.0 | Just right | Small | Small | Bottoms | ModCloth | 2013 | 0 |

### *1.1 ANALISI DELLE VARIABILI QUANTITATIVE*

```
dfc.describe()
```

| | item_id | rating | size | year | split |
|---|---|---|---|---|---|
| count | 99893.000000 | 99893.000000 | 78133.000000 | 99893.000000 | 99893.000000 |
| mean | 83944.091208 | 4.209594 | 3.217322 | 2015.476450 | 0.358564 |
| std | 43558.135979 | 1.080633 | 1.885934 | 1.970262 | 0.720993 |
| min | 6454.000000 | 1.000000 | 0.000000 | 2010.000000 | 0.000000 |
| 25% | 48629.000000 | 4.000000 | 2.000000 | 2014.000000 | 0.000000 |
| 50% | 77878.000000 | 5.000000 | 3.000000 | 2016.000000 | 0.000000 |
| 75% | 120126.000000 | 5.000000 | 4.000000 | 2017.000000 | 0.000000 |
| max | 155950.000000 | 5.000000 | 8.000000 | 2019.000000 | 2.000000 |

Nell'immagine sono riportate alcune delle principali statistiche descrittive per le variabili del dataset.

Si nota già come il valore medio dei rating è di gran lungo positivo (4.2), a dimostrare una soddisfazione generale da parte dei quasi 100000 clienti.

L'anno di produzione più frequente dei capi venduti è il 2015 in media, con delle taglie medie di valore 3.22 circa.

### *1.2 CREAZIONE DI UN DATASET RIDOTTO*

Molte variabili sono superflue per le future elaborazioni, quindi sono state tenute nel nuovo dataset solamente le variabili "item_id, user_id, rating_category". Anche qui c'è stato un controllo sui valori mancanti, ma in generale la riduzione del dataset ha aiutato di molto gli sforzi computazionali.

Queste alcune informazioni:

```
Number of ratings: 99892
Number of unique categories : 4
Number of unique users: 44783
Average ratings per user: 2.23
Average ratings per categories: 24973.0
```

Si può vedere il numero totale di rating, il numero di categorie, che in questo caso sono 4. Il numero di utenti, minore dei rating, a dimostrazione del fatto che un utente ha effettuato più valutazioni, 2.23 in media, come riportato. L'ultimo dato mostra che ci sono stati 24973 rating per categoria.

### 1.3 *ANALISI DELLA TENDEZZA CENTRALE DEI RATING*

L'analisi della tendenza centrale di una variabile è un'importante misura statistica che fornisce informazioni sul valore centrale o tipico di un insieme di dati.

Media e Mediana sono due tipiche misure di tendenza centrale che sono state analizzate:

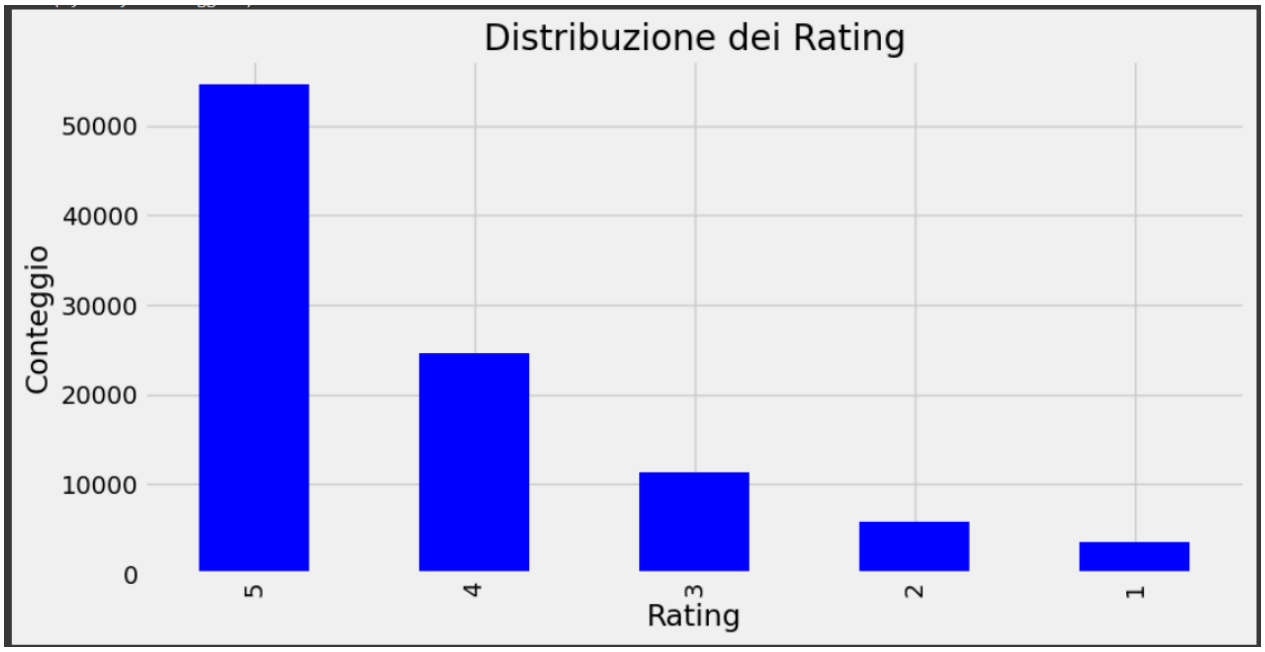
La media è la somma di tutti i valori divisa per il numero di osservazioni. È calcolata aggiungendo tutti i valori insieme e poi dividendo per il numero totale di valori.

La mediana è il valore centrale di un insieme di dati ordinato in modo crescente o decrescente. Per calcolare la mediana, i dati sono stati ordinati in modo decrescente e quindi si è individuato il valore esattamente al centro.

```
df['rating'].describe()
```

```
count    99892.000000
mean         4.209606
std          1.080632
min          1.000000
25%          4.000000
50%          5.000000
75%          5.000000
max          5.000000
Name: rating, dtype: float64
```

Sopra i risultati, con media di circa 4.21 del valore dei rating, e mediana (50esimo percentile) pari a 5



I risultati grafici confermano che i valori maggiormente assegnati ai rating sono di 4 e 5, indicando una tendenza positiva delle valutazioni dei clienti.

La presenza di valori bassi però non è da lasciare da parte, cosi da dimostrare la soggettività generale della vasta gamma di clienti, che però in questo caso sembra molto soddisfatta dai capi di abbigliamento acquistati.

Sarebbe utile, tramite anche valutazioni scritte e tramite web scraping, andare a capire le cose che non vanno bene ai clienti, così da migliorarle o migliorare generalmente la macroarea di interesse; ad esempio se c'è una concentrazione di recensioni negativa sulla categoria "Bottoms" andare a scavare nelle recensioni per capire il problema.

### *1.4 ANALISI DELLA DISPERSIONE DEI RATING*

L'analisi della dispersione di una variabile è un'indagine statistica che mira a valutare quanto i dati si discostano tra loro. La dispersione fornisce informazioni sulla variabilità dei dati e sulla distribuzione dei valori rispetto alla loro tendenza centrale.

In questo caso sono state valutate la varianza, la deviazione standard e la deviazione assoluta mediana.

La varianza e la deviazione standard sono due misure di dispersione che valutano quanto i dati si discostano dalla media.

La varianza indica la variabilità complessiva dei dati, invece la deviazione standard offre una stima della dispersione media dei dati rispetto alla media.

La deviazione assoluta invece valuta la variabilità dei dati rispetto alla mediana a differenza della deviazione standard, che si basa sulla media.

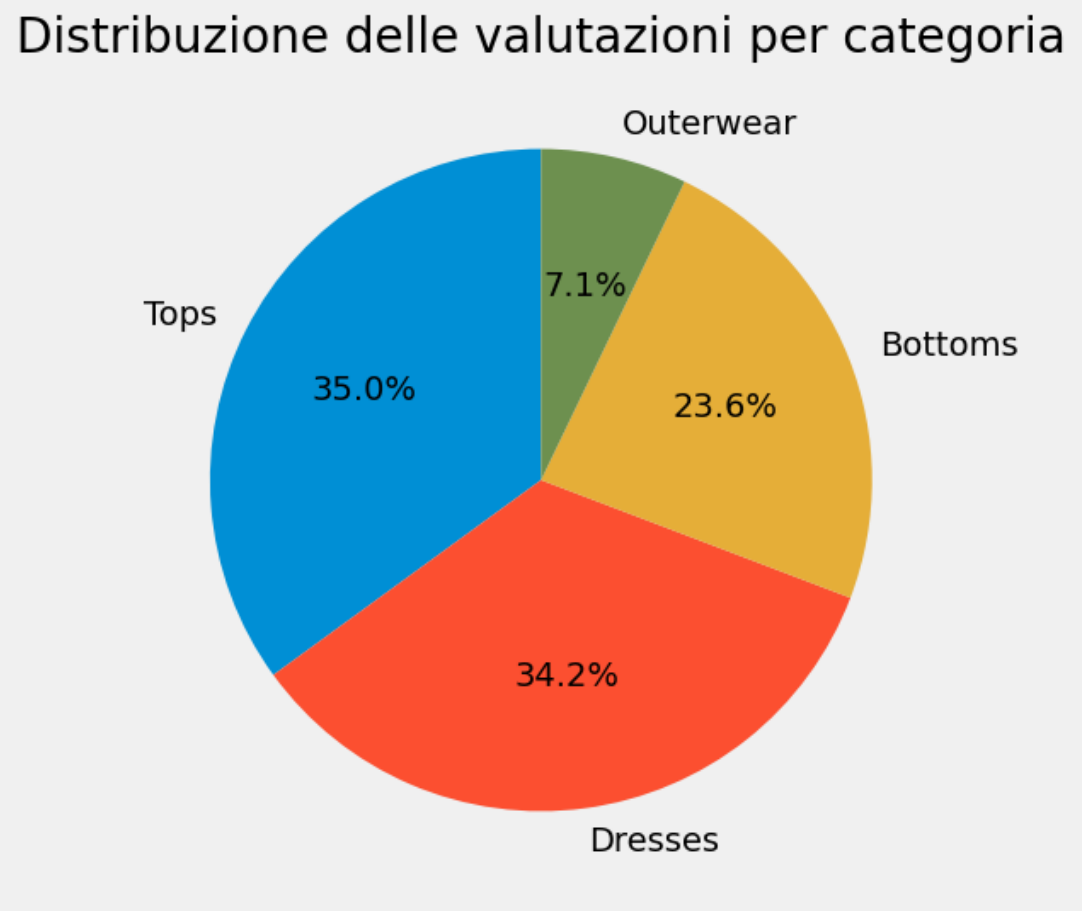
```
Varianza: 1.1677654816015504
Deviazione standard: 1.0806319824998474
Devianza assoluta mediana: 0.8635366062403818
```

### *1.5 ANALISI DEI RATING NELLE QUATTRO CATEGORIE*

Fin ora sono stati visti i rating in modo generale, ma ora ci si concentra sulle 4 categorie dei capi di abbigliamento acquistati dai clienti di ModCloth.

```
df.groupby('category')['rating'].count().sort_values(ascending=False).head()
```

```
category
Tops          34977
Dresses       34160
Bottoms       23624
Outerwear      7131
Name: rating, dtype: int64
```

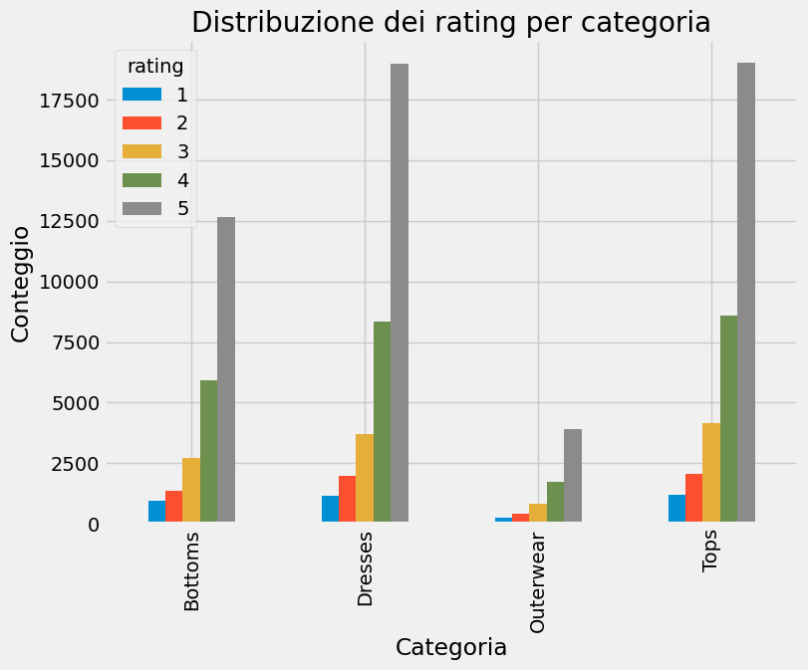



Già dall'output della groupby è abbastanza chiaro, ma basta vedere il grafico per capire su cosa si sono incentrate maggiormente gli acquisti e quindi le valutazioni dei clienti. Si nota come i "Dresses" e i "Tops" si spartiscono quasi a pari merito le fette maggiori, terzo posto per i "Bottoms" e minori rating generali per gli "Outerwear".

Generalmente questi dati seguono le tendenze di moda contemporanee, dove gli acquisti sono più incentrati su magliette e vestiti ad esempio, rispetto a pantaloni e scarpe, anche se non di moltissimo, per poi finire raramente su outerwear come sciarpe o coprispalle.

Di seguito si può vedere la distribuzione delle valutazioni per ogni categoria, che combaciano con le analisi dei rating fatte prima, infatti si può vedere come le “4 e 5 stelle” prevalgono generalmente in tutte e 4 le categorie.

```
category    rating
Bottoms     1          942
            2         1378
            3         2705
            4         5935
            5        12664
Dresses     1         1154
            2         1980
            3         3694
            4         8340
            5        18992
Outerwear   1          238
            2          413
            3          822
            4         1746
            5         3912
Tops        1         1189
            2         2080
            3         4138
            4         8570
            5        19000
Name: rating, dtype: int64
```




| | user_id | n_ratings |
|---|---|---|
| 5756 | Sarah | 250 |
| 411 | Amanda | 204 |
| 22660 | jennifer | 198 |
| 3735 | Laura | 197 |
| 2907 | Jennifer | 191 |
| 3333 | Katie | 188 |
| 38602 | sarah | 187 |
| 5353 | Rachel | 184 |
| 2949 | Jessica | 184 |
| 4482 | Megan | 177 |

L’output di fianco mostra gli utenti che hanno fornito il numero maggiore di valutazioni all’interno del dataset

### *1.6 CATEGORIE CON I RATING PIU' ALTI E PIU' BASSI*

Inizialmente cerco il valore medio dei rating per ogni categoria:

```
mean_rating = df.groupby('category')[['rating']].mean()
print(mean_rating)
```

```
              rating
category
Bottoms     4.185278
Dresses     4.230562
Outerwear   4.217361
Tops        4.203991
```

Dopo conto il numero di rating per ogni utente e per ogni articolo:

```
#RATING PER USER_ID
df.groupby('user_id')['rating'].count().describe()
```

```
count    44783.000000
mean         2.230579
std          6.548969
min          1.000000
25%          1.000000
50%          1.000000
75%          2.000000
max        250.000000
Name: rating, dtype: float64
```

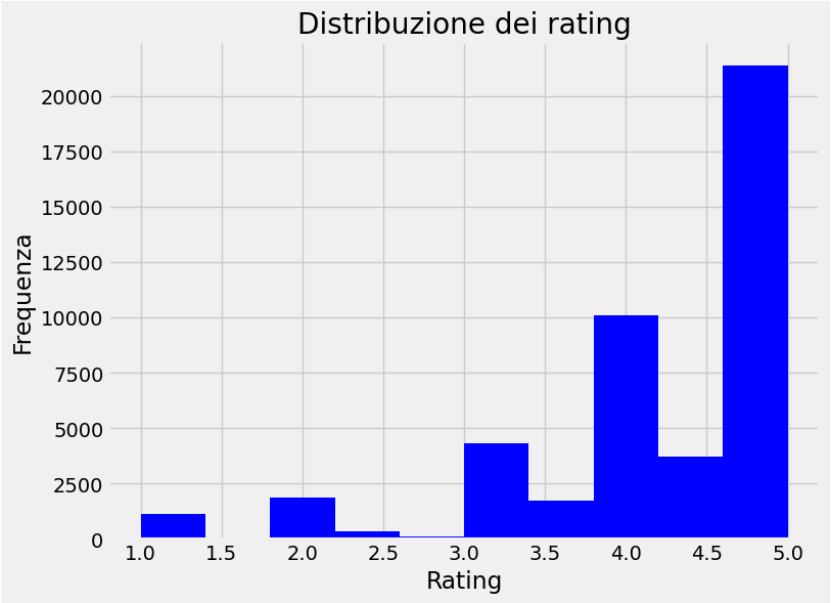
```
#RATING PER ITEM_ID
df.groupby('item_id')['rating'].count().describe()
```

```
count    1020.000000
mean       97.933333
std       216.416088
min         1.000000
25%         8.750000
50%        17.000000
75%        66.000000
max      1887.000000
Name: rating, dtype: float64
```

Analizzando i dati negli output sopra riportati, si vede come mediamente i rating più alti vengono accostati alla categoria "Dresses", ma generalmente le medie sono molto simili, 4.18 la più bassa fino a 4.23.

Negli ultimi due output si vedono i conteggi dei rating per utente e per articolo, con una media di 2.23 rating per cliente e di quasi 98 rating ogni articolo mediamente.

### *1.7 ANALISI CATEGORIA CON IL VALORE MEDIO DEI RATING MINORE (BOTTOMS)*



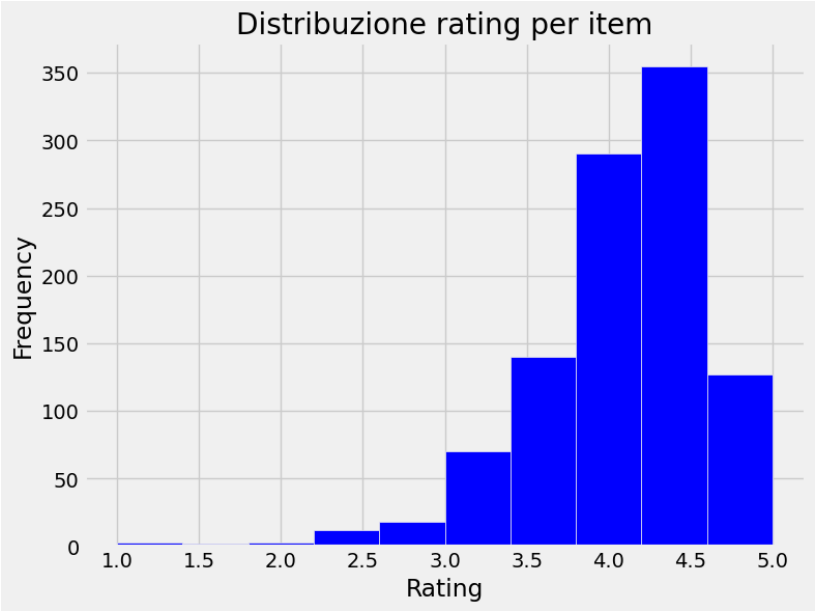

In questo grafico è possibile vedere come si distribuiscono i rating nella categoria "Bottoms" in base al loro conteggio.

Però in generale i grafici di tutte e quattro le categorie risulterebbero combacianti in quanto le medie viste nel punto 1.6 sono molto simili.

### *1.8 VALUTAZIONE DEI RATING MEDI PER OGNI ARTICOLO*

Nei seguenti grafici si vede la media dei rating per ogni item acquistato su ModCloth:

| item_id | rating |
|---|---|
| 6454 | 3.818182 |
| 7443 | 3.772502 |
| 11960 | 4.033958 |
| 16411 | 3.668254 |
| 21296 | 4.171760 |
| ... | ... |
| 155308 | 4.700000 |
| 155317 | 4.575758 |
| 155537 | 4.090909 |
| 155597 | 3.285714 |
| 155950 | 4.000000 |




## *2. CONFIGURAZIONE OTTIMALE DELL' ALGORITMO K-NN PER LA PREDIZIONE DEI RATING*

L'algoritmo K-NN (K-Nearest Neighbors) è un semplice algoritmo di apprendimento supervisionato utilizzato per la classificazione e la regressione. È ampiamente utilizzato in diversi campi come l'apprendimento automatico, l'intelligenza artificiale e l'analisi dei dati.

L'idea di base dell'algoritmo K-NN è abbastanza semplice: per classificare un nuovo punto dati o per stimare un valore di output, l'algoritmo cerca i K punti dati più vicini nel set di addestramento e utilizza la maggioranza delle etichette di classe o i valori di output dei K punti più vicini per fare una previsione.

Per identificare la configurazione ottimale dell'algoritmo K-NN per la predizione dei rating, è necessario testare diverse combinazioni di distanza, valore di K e approcci user-based o item-based. Per valutare le prestazioni di ciascuna configurazione, sono state usate le metriche di performance MSE (Mean Squared Error) e RMSE (Root Mean Squared Error). I passaggi chiave sono i seguenti:

- Preparazione dei dati
- Suddivisione del dataset in set di addestramento e set di test. Il set di addestramento sarà utilizzato per addestrare l'algoritmo K-NN, mentre il set di test sarà utilizzato per valutare le prestazioni dell'algoritmo

- Scelta del valore k: Determina il numero di vicini più vicini che verranno presi in considerazione durante la classificazione o la regressione. Questo valore, K, deve essere specificato inizialmente e può essere regolato per ottenere risultati migliori.
- Calcolo della distanza: K-NN utilizza la distanza tra le istanze per misurare la loro similarità. Il calcolo della distanza serve a trovare i K vicini più vicini all'istanza di input.
- Identificazione dei K vicini più vicini: Utilizzando la metrica di distanza scelta, vengono identificati i K vicini più vicini all'istanza di input nel set di dati di addestramento.
- Addestramento e valutazione: Per ogni combinazione dei parametri, addestra l'algoritmo K-NN utilizzando il set di addestramento e calcola le previsioni dei rating per il set di test. Quindi, calcola le metriche di performance come l'errore quadratico medio (MSE) e la radice dell'errore quadratico medio (RMSE) per valutare la qualità delle previsioni.
- Selezione della configurazione ottimale: Confronta le prestazioni delle diverse combinazioni utilizzando le metriche di performance. Identifica la combinazione di parametri che fornisce il valore più basso di MSE o RMSE, poiché indicano una migliore accuratezza delle previsioni dei rating.

### *2.1 CREAZIONE DATASET RIDOTTO*

```
fd = df[df.rating > 1].sort_values(by = 'rating' , ascending=False)
fd.head()
```

| | item_id | user_id | rating | category |
|---|---|---|---|---|
| **82343** | 153467 | sherry | 5 | Tops |
| **41313** | 77949 | Valerie | 5 | Bottoms |
| **41284** | 27439 | Natalie | 5 | Outerwear |
| **41285** | 115417 | chelseaevusher | 5 | Bottoms |
| **77317** | 153643 | B | 5 | Dresses |

```
fd.tail()
```

| | item_id | user_id | rating | category |
|---|---|---|---|---|
| **34511** | 16411 | kelseygordon13 | 2 | Dresses |
| **72973** | 121554 | dangermouserocks | 2 | Bottoms |
| **72974** | 44261 | dangermouserocks | 2 | Tops |
| **72976** | 84436 | eillamarie | 2 | Tops |
| **51427** | 82288 | shakenbaker7 | 2 | Dresses |

E’ stato ora creato un nuovo dataset che:

contiene solo le righe in cui il valore nella colonna 'rating' è maggiore di 1 ed è ordinato in base alla colonna 'rating' in ordine decrescente.

Una volta fatto ciò estraggo le due array contenenti gli ID unici degli utenti e degli articoli presenti nel DataFrame 'df' e riduco le dimensioni generali impostando a 300 gli item più votati e a 15000 gli utenti più votanti.

Da questi infine viene creato il dataset ridotto fd_mid contenente 4 colonne e 63837 righe.

```
[ ]  users_ids = df['user_id'].unique()
     items_ids = df['item_id'].unique()

RIDUCO IL DATASET

[ ]  most_rated_items = fd.item_id.value_counts().head(300).index
     most_rating_users = fd.user_id.value_counts().head(20000).index

     fd_mid = fd[df.item_id.isin(most_rated_items)]
     fd_mid = fd_mid[fd_mid.user_id.isin(most_rating_users)]

[ ]  fd_mid.size

     255348

[ ]  print(fd.shape)
     print(fd_mid.shape)

     (96369, 4)
     (63837, 4)
```

### *2.2 GRID SEARCH*

Grid search è una tecnica utilizzata nell'apprendimento automatico per trovare la migliore combinazione di iperparametri per un determinato algoritmo. Consiste nel definire una griglia di valori possibili per ogni iperparametro e testarli tutti in modo esaustivo per identificare la combinazione che produce le migliori prestazioni del modello.

- **Definizione della griglia dei parametri**: Le configurazioni prese in considerazione sono state le seguenti:
  ▪ Numero di vicini (N): Sono state testate configurazioni di N vicini nell'intervallo da 1 a 100.
  Questo parametro determina il numero di vicini da considerare per effettuare le raccomandazioni.
  ▪ Recommender Type: È stata valutata la scelta tra user-based e item-based. Questo parametro indica se il modello deve basarsi sulla somiglianza tra utenti o tra articoli per generare le raccomandazioni.
  ▪ Metrica di similarità: Sono state prese in considerazione due metriche di similarità, ovvero la Pearson correlation e la cosine similarity.

E’ stato selezionato un campione casuale del 20% dei dati da fd_mid che contiene le colonne 'user_id', 'item_id' e 'rating’.

```
ds_gridset = Dataset.load_from_df(fd_mid.sample(frac=0.2)[['user_id', 'item_id', 'rating']], reader)

param_grid = {
    'k': [5, 10, 15, 20, 25],
    'sim_options': {'name': ['cosine', 'pearson'],'user_based': [True, False]} }

gs = GridSearchCV(KNNBasic, param_grid, measures=['rmse', 'mse'])
gs.fit(ds_gridset)
```

- **Creazione delle combinazioni di parametri:** Genera tutte le possibili combinazioni dei valori dei parametri definiti nella griglia

```
Best RMSE = 0.9456
Best MSE = 0.8945
Best configuration = {'k': 5, 'sim_options': {'name': 'pearson', 'user_based': True}}
```

Questi risultati indicano che la configurazione ottimale del modello k-NN è stata trovata con un valore di k pari a 5 e utilizzando la similarità di Pearson con un approccio basato sugli utenti. La configurazione ottimale ha fornito un RMSE di 0.9456 e un MSE di 0.8945, che rappresentano le misure di errore medie più basse rispetto alle altre configurazioni testate.

- **Addestramento e valutazione:** Per ogni combinazione di parametri, addestramento del modello K-NN utilizzando il set di addestramento e valutazione delle prestazioni del modello utilizzando RMSE (Root Mean Square Error) e MSE (Mean Square Error). Utilizzo della Cross Validation.
- **Selezione della combinazione ottimale:** Confronta le prestazioni del modello per ciascuna combinazione di parametri e seleziona quella che produce i migliori risultati in termini di metrica di valutazione.
- **Valutazione finale:** Dopo aver selezionato la combinazione ottimale, valuta il modello utilizzando il set di test indipendente per avere un'idea finale delle sue prestazioni sulle nuove istanze non viste.

La **Cross-Validation** è una tecnica essenziale che viene spesso utilizzata durante l'addestramento e la valutazione della grid search per valutare le prestazioni del modello in modo robusto e ridurre il rischio di overfitting. Durante la grid search, ogni combinazione di parametri viene valutata attraverso la cross-validation per ottenere una stima accurata delle prestazioni del modello su dati non visti. La cross-validation divide il set di dati di addestramento in diverse parti chiamate fold, e ad

ogni iterazione, un fold diverso viene utilizzato come set di validazione mentre gli altri fold vengono utilizzati come set di addestramento. Di seguito i risultati:

| | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 | Mean | Std |
|---|---|---|---|---|---|---|---|
| RMSE (testset) | 0.9576 | 0.9548 | 0.9509 | 0.9323 | 0.9491 | 0.9489 | 0.0088 |
| MSE (testset) | 0.9171 | 0.9116 | 0.9042 | 0.8692 | 0.9008 | 0.9006 | 0.0167 |
| Fit time | 2.41 | 2.64 | 2.52 | 2.67 | 2.79 | 2.61 | 0.13 |
| Test time | 0.08 | 0.08 | 0.08 | 0.13 | 0.08 | 0.09 | 0.02 |

Guardando l'output, i valori sembrano simili a quelli ottenuti prima, basta vedere la media del RMSE (tested) pari a 0.9489 e quella del MSE (tested) uguale a 0.9006. Complessivamente i risultati della cross-validation suggeriscono che il modello ha una buona capacità di fare previsioni accurate sui dati di test, con buoni tempi di adattamento e di test.

### *2.3 PREDIZIONE DELL'ALGORITMO KNNBasic*

Per prima cosa fitto il modello su tutto il dataset, ovvero addestro il modello utilizzando tutte le coppie utente-item con i rispettivi rating disponibili nel dataset.

```
ds = Dataset.load_from_df(fd_mid[['user_id', 'item_id', 'rating']], reader)

# Costruisco il training set
trainset = ds.build_full_trainset()

algo = KNNBasic()

algo.fit(trainset)
```

**KNNBasic** è un algoritmo di filtraggio collaborativo basato sulla vicinanza nei sistemi di raccomandazione. L'algoritmo KNNBasic utilizza la similarità tra utenti o elementi per effettuare previsioni sui rating mancanti o per raccomandare nuovi elementi agli utenti. Ecco il funzionamento dell'algoritmo KNNBasic:

1. Calcolo della similarità: L'algoritmo calcola la similarità tra gli utenti o gli elementi utilizzando una metrica di similarità, come la similarità del coseno o la correlazione di Pearson. La similarità viene calcolata confrontando i rating dati dagli utenti o le caratteristiche degli elementi.

2. Selezione dei vicini: Per un utente o un elemento target, l'algoritmo seleziona i K vicini più simili in base alla similarità calcolata. Questi vicini saranno utilizzati per effettuare previsioni o raccomandazioni.

3. Ponderazione dei rating: L'algoritmo assegna un peso ai rating dei vicini in base alla loro similarità con l'utente o l'elemento target. I vicini più simili ricevono un peso maggiore nel calcolo delle previsioni o delle raccomandazioni.

4. Calcolo delle previsioni: Utilizzando i rating ponderati dei vicini, l'algoritmo calcola una previsione per un rating mancante o raccomanda nuovi elementi all'utente target.

```
import random

for _ in range(10):
    sample_row = fd_mid.sample().iloc[0]
    user_id = sample_row['user_id']
    item_id = sample_row['item_id']

    prediction = algo.predict(user_id, item_id, r_ui=2, verbose=True)
    estimated_rating = prediction.est

    print(f"User: {user_id}\tItem: {item_id}\tEstimated Rating: {estimated_rating}")
```

```
user: josephine.grahm item: 65046      r_ui = 2.00   est = 5.00   {'actual_k': 40, 'was_impossible': False}
User: josephine.grahm    Item: 65046     Estimated Rating: 5
user: sarah.j.hashmi item: 54887      r_ui = 2.00   est = 4.00   {'actual_k': 40, 'was_impossible': False}
User: sarah.j.hashmi     Item: 54887     Estimated Rating: 4.0
user: karen.w.linder item: 48629      r_ui = 2.00   est = 4.00   {'actual_k': 40, 'was_impossible': False}
User: karen.w.linder     Item: 48629     Estimated Rating: 4.0
user: nicole     item: 142983     r_ui = 2.00   est = 4.86   {'actual_k': 40, 'was_impossible': False}
User: nicole     Item: 142983    Estimated Rating: 4.864261450026822
user: anderson.kelly item: 34935      r_ui = 2.00   est = 5.00   {'actual_k': 40, 'was_impossible': False}
User: anderson.kelly     Item: 34935     Estimated Rating: 5
user: cpresti    item: 68332      r_ui = 2.00   est = 4.00   {'actual_k': 40, 'was_impossible': False}
User: cpresti    Item: 68332     Estimated Rating: 4.0
user: christinia item: 31752      r_ui = 2.00   est = 4.00   {'actual_k': 40, 'was_impossible': False}
User: christinia         Item: 31752     Estimated Rating: 4.0
user: ascherlm   item: 129278     r_ui = 2.00   est = 3.00   {'actual_k': 40, 'was_impossible': False}
User: ascherlm   Item: 129278    Estimated Rating: 3.0000000000000013
user: jennfuquay item: 40899      r_ui = 2.00   est = 4.00   {'actual_k': 40, 'was_impossible': False}
User: jennfuquay         Item: 40899     Estimated Rating: 4.0
user: sharon     item: 115417     r_ui = 2.00   est = 5.00   {'actual_k': 40, 'was_impossible': False}
User: sharon     Item: 115417    Estimated Rating: 5
```

E’ stato considerato un numero variabile di vicini più vicini (indicato come 'actual_k') per ogni previsione.

Dall’output è possibile vedere ad esempio l’User: jennfuquay Item: 40899 Estimated Rating: 4.0

Il rating stimato per questa coppia utente-elemento è 4.0, che indica un sentimento positivo verso l'elemento. L'algoritmo ha considerato 40 vicini più prossimi per fare questa previsione.

## 3. FILLING DELLA MATRICE DI RATING CON LA CONFIGURAZIONE OTTIMALE

La "matrice dei rating" si riferisce a una rappresentazione tabellare dei rating o delle valutazioni fornite dagli utenti o dai clienti a una serie di elementi o oggetti. È spesso utilizzata in contesti come la raccomandazione di prodotti o servizi.

Nella matrice dei rating, le righe rappresentano i clienti, le colonne rappresentano gli articoli e le celle contengono i rating attribuiti dagli utenti a ciascun capo d'abbigliamento.

Il lavoro è stato diviso in due fasi:

```
users_subset = users_ids[:1500]

rating_matrix = np.zeros((len(users_subset), len(items_ids)))
is_predicted = np.zeros((len(users_subset), len(items_ids)))

fd_mid.set_index(['user_id', 'item_id'], inplace=True)
```

Il codice crea un sottoinsieme di utenti, inizializza una matrice dei rating e una matrice di tracking dei valori predetti, e modifica il DataFrame di origine impostando l'indice su due colonne specifiche.

La seconda fase invece:

```
for i, user in enumerate(users_subset):
    for j, item in enumerate(items_ids):

        if (user, item) in fd_mid.index:
            rating_matrix[i, j] = fd_mid.loc[user, item]['rating']

        else:
            rating_matrix[i, j] = round(algo.predict(user, item).est)
            is_predicted[i, j] = 1

fd_mid.reset_index(inplace=True)
```

In sintesi, il codice cicla su ogni utente e oggetto nel sottoinsieme specificato, verifica se è disponibile un rating noto nel DataFrame fd_mid, imposta il rating corrispondente nella matrice dei rating rating_matrix, altrimenti fa una previsione utilizzando l'oggetto algo e imposta il rating predetto nella matrice. La matrice is_predicted tiene traccia dei valori predetti. Infine, il DataFrame fd_mid viene ripristinato al suo stato originale.

Riassumendo, il primo codice prepara le strutture dati necessarie per il processo di filling dei dati, mentre il secondo codice riempie la matrice dei rating con i valori noti presenti nel DataFrame fd_mid e con le previsioni dei valori mancanti utilizzando un oggetto algo. Alla fine del processo, la matrice dei rating rating_matrix sarà

completata con valori noti e previsioni, mentre la matrice is_predicted terrà traccia dei valori predetti. I risultati sono:

### *3.1 MATRICE DI RATING CON GLI 0*

```
is_predicted[:10, :10]
```

```
array([[0., 1., 1., 0., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [0., 1., 1., 0., 0., 1., 1., 1., 0., 0.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [0., 1., 0., 1., 1., 1., 1., 1., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [0., 1., 0., 0., 1., 1., 1., 0., 1., 1.],
       [1., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [0., 1., 0., 0., 1., 1., 1., 1., 0., 1.],
       [0., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])
```

Nella matrice is_predicted, i valori 1 e 0 vengono utilizzati per indicare se un rating è stato previsto o se è un valore noto. Ecco cosa significano i valori 1 e 0 in questa matrice:

**Valore 1**: Indica che il rating corrispondente nella matrice dei rating rating_matrix è stato predetto utilizzando un algoritmo o un modello. Questo significa che il valore presente in rating_matrix è un'ipotesi o una stima del rating reale mancante.

**Valore 0**: Indica che il rating corrispondente nella matrice dei rating rating_matrix è un valore noto, ossia è presente nel DataFrame fd_mid. Questi sono i rating effettivi che sono stati raccolti o osservati.

Quindi, la matrice is_predicted viene utilizzata per tenere traccia dei valori predetti nella matrice dei rating. I valori 1 in is_predicted indicano le posizioni in rating_matrix in cui sono presenti le previsioni, mentre i valori 0 indicano le posizioni in cui sono presenti i valori noti. Questo può essere utile per distinguere tra valori effettivi e valori stimati nel processo di analisi o modellazione successivo.

### *3.2 MATRICE DEI RATING CON VALORI NOTI E PREVISIONI*

```
rating_matrix[:10, :10]
```

```
array([[4., 4., 5., 5., 5., 5., 4., 5., 5., 5.],
       [4., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
       [4., 5., 5., 5., 4., 5., 4., 5., 5., 5.],
       [4., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
       [4., 4., 3., 5., 4., 4., 4., 5., 4., 5.],
       [4., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
       [2., 5., 5., 5., 5., 5., 5., 5., 5., 5.],
       [4., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
       [4., 5., 3., 5., 4., 5., 4., 5., 3., 5.],
       [4., 5., 5., 5., 5., 5., 4., 5., 4., 5.]])
```

Nella matrice rating_matrix, i valori rappresentano i rating assegnati dagli clienti agli articoli. Ogni valore nella matrice corrisponde al rating dato da un utente specifico a un capo specifico. Ad esempio, supponiamo di avere un utente i-esimo nella posizione di riga i e un oggetto j-esimo nella posizione di colonna j nella matrice. Il valore rating_matrix[i, j] rappresenta il rating assegnato dall'utente i-esimo all'oggetto j-esimo.

## *4. SEGMENTAZIONE DEGLI UTENTI IN BASE ALLE PREFERENZE: ALGORITMO DI CLUSTERING K-MEANS CON COSINE SIMILARITY*

La segmentazione degli utenti in base alle preferenze può essere realizzata utilizzando l'algoritmo di clustering K-means con la similarità del coseno come misura di similarità.

L'algoritmo di clustering K-means è uno dei più comuni e semplici algoritmi utilizzati per raggruppare i dati in cluster. Il suo obiettivo principale è suddividere un set di dati in k cluster, in cui k è un numero predefinito scelto dall'utente.

### *CREAZIONE MATRICE K-MEANS*

Creazione di una matrice matrice_kmeans_mid riempita con i valori di rating dai dati presenti nel dataframe fd_mid, tenendo conto solo degli utenti e degli oggetti selezionati e scartando gli utenti e gli oggetti senza valori di rating.

```
matrice_kmeans_mid[:10, :10]
```

```
array([[2., 0., 5., 5., 5., 5., 4., 5., 5., 2.],
       [4., 5., 4., 5., 4., 3., 5., 5., 5., 4.],
       [5., 4., 4., 5., 4., 2., 0., 5., 4., 4.],
       [5., 2., 5., 4., 5., 5., 0., 3., 4., 0.],
       [5., 5., 4., 5., 4., 5., 2., 5., 4., 5.],
       [4., 5., 3., 4., 2., 5., 5., 2., 0., 4.],
       [3., 5., 5., 4., 5., 3., 3., 5., 5., 5.],
       [5., 4., 3., 5., 5., 3., 3., 4., 4., 0.],
       [5., 5., 5., 5., 5., 5., 0., 5., 5., 4.],
       [5., 2., 5., 4., 3., 4., 5., 5., 3., 4.]])
```

### *4.1 DETERMINAZIONE DEL NUMERO K DI CLUSTER CON METODO ELBOW*

Il metodo del gomito, noto anche come **metodo elbow**, è una tecnica utilizzata per determinare il numero ottimale di cluster in un algoritmo di clustering, come il K-means in questo caso. Prende il nome dalla forma del grafico risultante, che assomiglia a un gomito o a una curva con un punto di flessione.

Questo metodo si basa sulla valutazione della variazione interna dei cluster, misurata tramite la metrica WCSS (Within-Cluster Sum of Squares).

La **metrica WCSS (Within-Cluster Sum of Squares)** è una misura utilizzata nel contesto dell'algoritmo di clustering per valutare la coerenza dei cluster ottenuti. La WCSS misura la somma dei quadrati delle distanze tra i punti all'interno di ciascun cluster e il centroide del cluster a cui appartengono.

Quando si esegue il K-means, l'obiettivo principale è creare cluster in cui i punti all'interno di ciascun cluster siano simili tra loro e distinti dagli altri cluster e la metrica WCSS viene utilizzata per quantificare quanto i punti di un cluster siano vicini al centroide del cluster stesso.

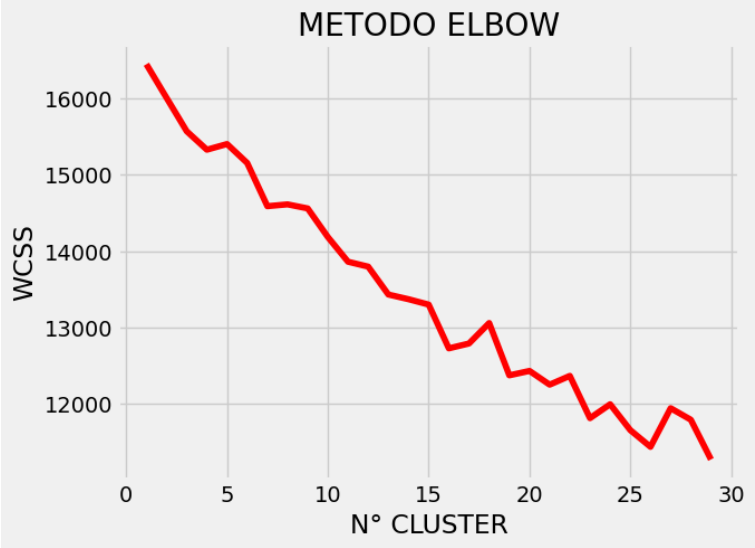
L'obiettivo è minimizzare la WCSS, poiché una WCSS più bassa indica che i punti all'interno di un cluster sono più vicini al centroide e quindi i cluster sono più coerenti. Il metodo del gomito utilizza la WCSS per determinare il numero ottimale di cluster, cercando il punto di flessione nel grafico della WCSS rispetto al numero di cluster.

### *4.2 VALORI WCSS E PRIMO GRAFICO USANDO matrice_kmeans_mid*

```
for i in range(1, 30):
    kclusterer = KMeansClusterer(i, distance=cosine_distance, avoid_empty_clusters=True)
    clusters = kclusterer.cluster(matrice_kmeans_mid, assign_clusters=True)
    wcss.append(get_wcss(matrice_kmeans_mid, clusters, kclusterer.means()))
    print('Numero di cluster:', i, 'WCSS:', wcss[-1])
```

```
Numero di cluster: 1 WCSS: 16447.227306305485
Numero di cluster: 2 WCSS: 16007.566951729403
Numero di cluster: 3 WCSS: 15570.001250891048
Numero di cluster: 4 WCSS: 15328.34305440445
Numero di cluster: 5 WCSS: 15402.956720090444
Numero di cluster: 6 WCSS: 15154.858991939664
Numero di cluster: 7 WCSS: 14589.28404228312
Numero di cluster: 8 WCSS: 14613.019703603546
Numero di cluster: 9 WCSS: 14559.926065190597
Numero di cluster: 10 WCSS: 14185.608316189262
```

Questo codice utilizza la matrice matrice_kmeans_mid, vista nel punto 4, per eseguire il clustering K-means e calcolare il WCSS. Questa matrice è stata generata a partire dal dataframe fd_mid, e contiene valori di rating per utenti e oggetti selezionati, con zeri per le posizioni corrispondenti a valori mancanti.




L'output è una lista wcss che contiene i valori di WCSS per diverse configurazioni di clustering K-means con un numero variabile di cluster.

Osservando i risultati, il valore scende all'aumentare del numero di cluster (sono riportati i primi 10 di un range da 1 a 30), però questa discesa è molto uniforme e non ci sono discese forti, come possiamo vedere nel grafico.

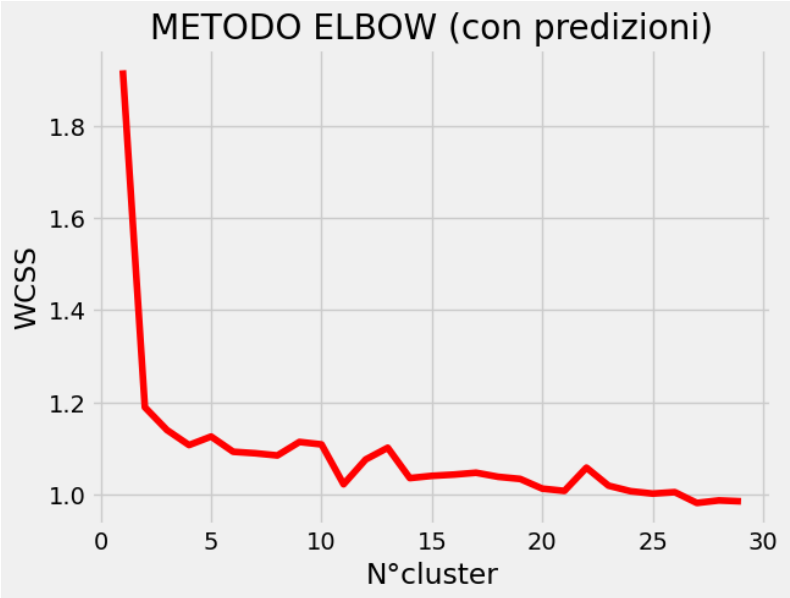
### *4.3 VALORI WCSS E GRAFICO A GOMITO USANDO rating_matrix*

```
wcss = []

for i in range(1, 30):
    kclusterer = KMeansClusterer(i, distance=cosine_distance, avoid_empty_clusters=True)
    clusters = kclusterer.cluster(rating_matrix, assign_clusters=True)
    wcss.append(get_wcss(rating_matrix, clusters, kclusterer.means()))
    print('Numero di cluster:', i, 'WCSS:', wcss[-1])
```

```
Numero di cluster: 1 WCSS: 1.9205815986017547
Numero di cluster: 2 WCSS: 1.1892695932980635
Numero di cluster: 3 WCSS: 1.1398192189052019
Numero di cluster: 4 WCSS: 1.107160597357742
Numero di cluster: 5 WCSS: 1.1256582405108138
Numero di cluster: 6 WCSS: 1.0926155433892706
Numero di cluster: 7 WCSS: 1.0893671581201236
Numero di cluster: 8 WCSS: 1.0845787136462233
Numero di cluster: 9 WCSS: 1.1139155305222752
Numero di cluster: 10 WCSS: 1.108558158039685
```

Questo codice utilizza la matrice rating_matrix per eseguire il clustering K-means e calcolare il WCSS. Ora i risultati sono ben diversi; c'è una netta differenza fra il cluster 1 e il cluster 2: si passa da 1.92 a 1.18 e questo cambia di molto il grafico, rendendolo decisamente a gomito.




L'obiettivo del grafico del metodo del gomito è individuare il punto di flessione, che rappresenta il numero ottimale di cluster. Questo è il punto in cui l'aggiunta di ulteriori cluster non contribuisce significativamente a ridurre il valore di WCSS. Quindi, il numero di cluster corrispondente al punto di flessione viene considerato come il numero ottimale di cluster per la suddivisione dei dati e in questo caso si vede benissimo che il punto di flessione è 2.

### *4.4 ESEGUO IL K-MEANS (k=2 cluster)*

```
n_clusters = 2

kmeans_clusterer = KMeansClusterer(n_clusters, distance=cosine_distance, avoid_empty_clusters=True)

clusters = kmeans_clusterer.cluster(rating_matrix, assign_clusters=True)
users_clusters = dict(zip(users_ids, clusters))
```

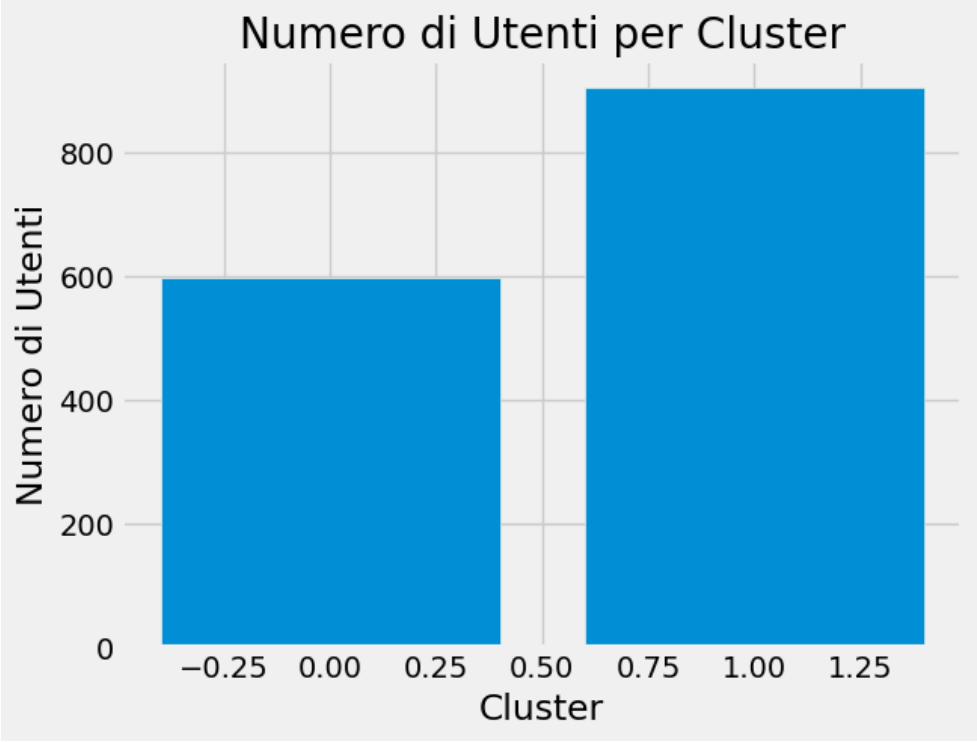
Il codice esegue il clustering K-means sulla matrice di rating rating_matrix utilizzando 2 cluster. Il risultato finale è un dizionario che mappa gli utenti ai loro rispettivi cluster. Questo dizionario fornisce informazioni sulla segmentazione degli utenti in base alle loro preferenze, ottenute attraverso l'algoritmo di clustering K-means:

```
users_clusters
```

```
{'Alex': 1,
 'carolyn.agan': 0,
 'Robyn': 1,
 'De': 0,
 'tasha': 1,
 'gina.chihos': 0,
 'Kim': 1,
 'jess.betcher': 0,
 'Elissa': 1,
 'Yvette': 1,
```

Di fianco una piccola parte del dizionario creato, i due cluster sono stati chiamati “Cluster 0” e “Cluster 1” e queste sono le loro dimensioni.

```
Cluster 0 utenti: 596
Cluster 1 utenti: 904
```



## 5. CREAZIONE PER OGNI UTENTE DELLA LISTA DEGLI N ITEMS DA CONSIGLIARE

```
def get_top_n_item(user, item, rating_matrix, is_predicted, n=10):

    items_suggeriti = {}

    for i, user_id in enumerate(user):

        sorting_rating_index = np.argsort(rating_matrix[i])[::-1]
        sorting_rating_index = sorting_rating_index[is_predicted[i][sorting_rating_index] == 1]

        items_suggeriti [user_id] = item[sorting_rating_index][:n]


    items_suggeriti  = pd.DataFrame.from_dict(items_suggeriti , orient='index')
    items_suggeriti .columns = ['ITEM_' + str(i) for i in range(1, n+1)]
    items_suggeriti .index.name = 'USER_ID'

    return items_suggeriti

get_top_n_item(users_subset, items_ids, rating_matrix, is_predicted, n=10).head(5)
```

La funzione get_top_n_item prende in input una lista di utenti (user), una lista di elementi (item), una matrice di rating (rating_matrix) e una matrice che indica se le valutazioni sono state previste o meno (is_predicted). La funzione restituisce un DataFrame contenente gli N elementi migliori da suggerire per ogni utente, in questo caso 10. Ecco alcuni risultati, che mostrano il cliente e l'ID dell'articolo consigliato:

| USER_ID | ITEM_1 | ITEM_2 | ITEM_3 | ITEM_4 | ITEM_5 | ITEM_6 | ITEM_7 | ITEM_8 | ITEM_9 | ITEM_10 |
|---|---|---|---|---|---|---|---|---|---|---|
| Alex | 102736 | 142983 | 82288 | 153081 | 123500 | 85507 | 132261 | 143686 | 153709 | 84436 |
| carolyn.agan | 153228 | 106348 | 135264 | 151263 | 150583 | 150788 | 150982 | 137816 | 150984 | 150712 |
| Robyn | 82288 | 141880 | 153081 | 145278 | 108981 | 141845 | 110169 | 143136 | 113512 | 104356 |
| De | 153228 | 106348 | 135264 | 151263 | 150583 | 150788 | 150982 | 137816 | 150984 | 150712 |
| tasha | 116313 | 110169 | 107794 | 141072 | 126821 | 153454 | 108981 | 153881 | 113643 | 106242 |

## 6. FILLING DELLA MATRICE DI RATING ATTRAVERSO MATRIX FACTORIZATION (MF) IN AGGIUNTA A K-NN E CONFRONTO DEI RISULTATI OTTENUTI IN TERMINI DI MSE E RMSE

La Matrix Factorization, o Fattorizzazione della Matrice, è una tecnica utilizzata per la raccomandazione e la previsione dei rating in un sistema di raccomandazione.

L'idea principale della Matrix Factorization è approssimare la matrice di rating con il prodotto di due matrici più piccole: una matrice degli utenti e una matrice degli oggetti.

L'obiettivo della Matrix Factorization è trovare le matrici degli utenti e degli oggetti che approssimano al meglio la matrice di rating originale. Questo viene solitamente fatto minimizzando l'errore quadratico medio (MSE) e l'errore quadratico medio radice (RMSE), tra la matrice di rating originale e la sua approssimazione basata sul prodotto delle matrici degli utenti e degli oggetti.

Una volta addestrate le matrici degli utenti e degli oggetti, è possibile effettuare previsioni sui rating mancanti nella matrice di rating originale calcolando il prodotto tra la riga corrispondente della matrice degli utenti e la colonna corrispondente della matrice degli oggetti.

### 6.1 GRID SEARCH 2.0

Viene eseguita la grid search utilizzando l'algoritmo SVD come modello e le metriche di valutazione "RMSE" e "MSE". La grid search cerca la combinazione ottimale di iperparametri che minimizza l'errore RMSE o MSE. Di seguito i risultati:

```
Best RMSE = 0.8945
Best MSE = 0.8002
Best configuration = {'lr_all': 0.005, 'reg_all': 0.06, 'n_epochs': 10, 'n_factors': 25}
```

Successivamente viene eseguita una **cross validation** a 5 fold che restituisce informazioni sulle prestazioni dell'algoritmo SVD utilizzando i migliori parametri selezionati:

| | Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5 | Mean | Std |
|---|---|---|---|---|---|---|---|
| RMSE (testset) | 0.9296 | 0.8985 | 0.9085 | 0.9009 | 0.9171 | 0.9109 | 0.0114 |
| MSE (testset) | 0.8642 | 0.8073 | 0.8255 | 0.8116 | 0.8411 | 0.8299 | 0.0208 |
| Fit time | 0.04 | 0.05 | 0.05 | 0.06 | 0.07 | 0.05 | 0.01 |
| Test time | 0.02 | 0.04 | 0.01 | 0.01 | 0.02 | 0.02 | 0.01 |

### *6.2 TRAINSET E ADDESTRAMENTO DEL MODELLO SVD*

```
ds_mf = Dataset.load_from_df(fd_mid[['user_id', 'item_id', 'rating']], reader)

trainset_mf = ds_mf.build_full_trainset()

algo_mf.fit(trainset_mf)
```

Con questo codice carico il dataset completo per la creazione della matrice di rating: Viene caricato l'intero DataFrame fd_mid per creare il dataset completo per la creazione della matrice di rating (ds_mf).

Successivamente c'è la creazione del trainset e addestramento dell'algoritmo SVD: Viene creato il trainset utilizzando il dataset completo ds_mf e viene addestrato l'algoritmo SVD algo_mf con il trainset.

### *6.3 RIEMPIMENTO DELLA MATRICE DEI RATING (MF)*

Viene creata la matrice di rating **rating_matrix_mf**. Se una valutazione esiste nel DataFrame fd, viene inserita nella matrice di rating; altrimenti, viene effettuata una previsione utilizzando l'algoritmo SVD e la previsione viene arrotondata. Viene inoltre creata la matrice **is_predicted_mf** per indicare quali valutazioni sono state previste.

```
is_predicted_mf[:10, :10]

array([[0., 1., 1., 0., 1., 1., 1., 1., 1., 1.],
       [0., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [0., 1., 1., 0., 0., 1., 1., 1., 0., 0.],
       [0., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [0., 1., 0., 1., 1., 1., 1., 1., 1., 1.],
       [0., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [0., 1., 0., 0., 1., 1., 1., 0., 1., 1.],
       [0., 1., 1., 1., 1., 1., 1., 1., 1., 1.],
       [0., 1., 0., 0., 1., 1., 1., 1., 0., 1.],
       [0., 1., 1., 1., 1., 1., 1., 1., 1., 1.]])
```

Un **valore di 1** nella matrice is_predicted_mf indica che il rating nella corrispondente posizione della matrice rating_matrix_mf è stato predetto o imputato dall'algoritmo utilizzato, mentre un **valore di 0** indica che il rating è stato effettivamente fornito dall'utente nella matrice di dati originale.

```
rating_matrix_mf[:10, :10]

array([[4., 4., 4., 5., 4., 4., 4., 4., 4., 4.],
       [3., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
       [4., 4., 4., 5., 4., 4., 4., 4., 5., 5.],
       [4., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
       [4., 4., 3., 4., 4., 4., 4., 4., 4., 4.],
       [5., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
       [2., 4., 5., 5., 4., 4., 4., 5., 4., 4.],
       [5., 4., 4., 4., 4., 4., 4., 4., 4., 4.],
       [4., 4., 3., 5., 4., 4., 4., 4., 3., 4.],
       [4., 4., 4., 4., 4., 4., 4., 4., 4., 4.]])
```

I valori nella matrice rating_matrix_mf indicano i rating assegnati agli articoli da parte degli utenti. Le righe corrispondano agli utenti e le colonne corrispondano agli oggetti. Ogni cella della matrice contiene un valore che rappresenta il rating assegnato dall'utente all'oggetto corrispondente.

## *CONCLUSIONI*

I sistemi di raccomandazione svolgono un ruolo importante nell'aiutare gli utenti a scoprire contenuti rilevanti e interessanti. Nel nostro caso, abbiamo utilizzato algoritmi sofisticati per combinare i dati degli utenti e le informazioni sugli oggetti al fine di creare un sistema di raccomandazione per gli utenti della piattaforma ModCloth. Questo sistema offre esperienze personalizzate e aiuta a mantenere gli utenti fedeli fornendo loro una lista di oggetti consigliati di alta qualità basata sulla matrice di rating finale.

Attraverso l'analisi del data dataset "Marketing Bias (ModCloth)", possiamo ottenere informazioni utili per migliorare la progettazione e la produzione di abbigliamento. Inoltre, questo ci aiuta a capire meglio le esigenze e i gusti dei consumatori nel settore della moda. In sostanza, i sistemi di raccomandazione offrono un vantaggio sia per gli utenti, che ottengono suggerimenti personalizzati, sia per le aziende, che possono migliorare la loro offerta e soddisfare meglio le preferenze dei clienti.